TELIPORT
Script Your Future
Season 4

TATA
TATA ELXSI

Case Study No. 5

Team Details

VehicleGPT Edge - On-Device SLM/LLM Vehicle Assistant

Team Name: <enter team name>

Members: <names and roll numbers>

College: <college name> | Email: <contact>

THINK. BUILD. CREATE. WIN

THE CHALLENGE

Software-defined vehicles produce answers no one can ask for

Real questions owners and service engineers ask today

- “Why has my driving range reduced over the last two weeks?”
- “When did fuel efficiency show the largest drop?”
- “Show all ADAS disengagement events from yesterday.”
- “What happened before the battery overheating warning?”
- “Compare energy consumption between city and highway driving.”

Data at the source: CAN / CAN-FD · SOME/IP · Android Vehicle HAL · QNX services & logs · ADAS · camera feeds · diagnostic logs

Today: dealer visit, dongle + laptop, or a cloud portal → days. **Target:** the same answer, in-vehicle, in under 2 seconds.

HARD CONSTRAINTS FROM THE CASE BRIEF

- **No cloud dependency**
  local-only inference, storage and search
- **~2 GB RAM, ~2 ARM cores**
  embedded-class hardware (Pi 4/5, automotive ARM, QEMU/Graviton)
- **Response < 2 seconds**
  for common natural-language queries
- **Weeks of history, searchable**
  at least several weeks retained on-device
- **Safety isolation**
  safety-critical functions stay outside the AI system

OUR SOLUTION · VEHICLEGPT EDGE

Evidence-first assistant: the model explains, the data proves

1 · SENSE & STORE

- Normalisers for CAN/CAN-FD, SOME/IP, Android Vehicle HAL, QNX logs and ADAS events
- One canonical time-series + event store: weeks of history on the vehicle
- CARLA drives and recorded-log replay make every scenario repeatable

2 · ASK & PROVE

- Local SLM (≤1.5B, 4-bit) interprets the question — it never invents the numbers
- SQL, temporal and anomaly engines build an evidence package per query
- Validator re-checks every figure and citation before anything is shown

3 · EXPLAIN & ACT

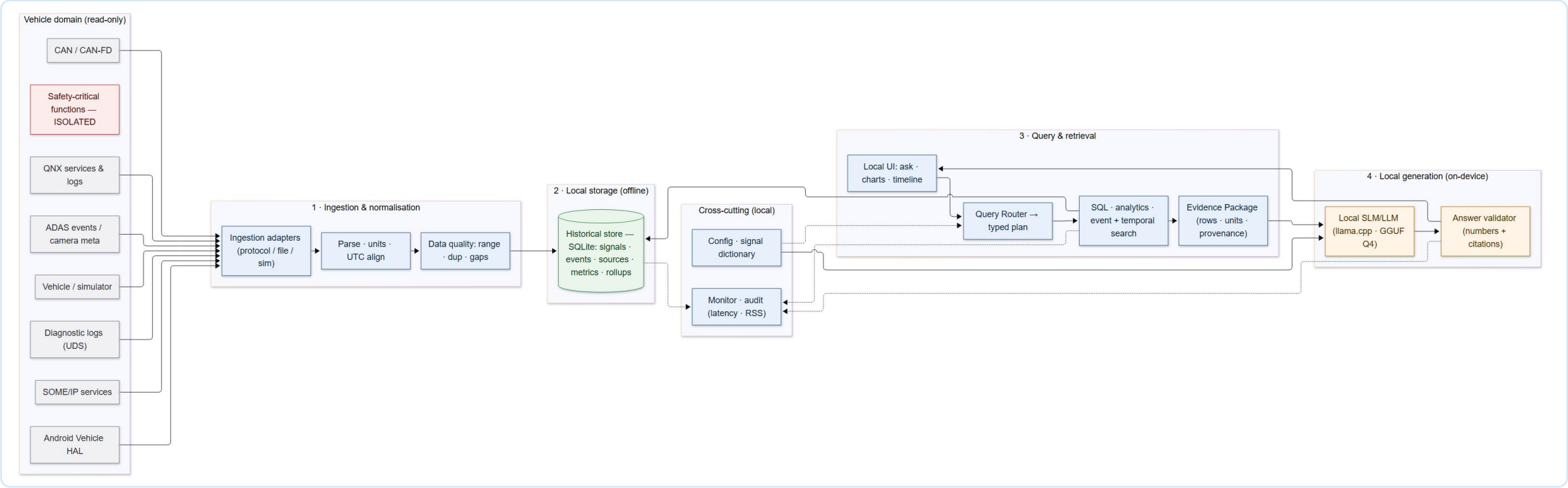
- Plain-language answer + trend chart on head unit or service tablet
- Exact lookups take a model-free fast path (typically < 200 ms)
- Read-only by design: no write path to actuators or safety functions

Who it serves: **Driver · Service engineer · Fleet operator · Validation & QA team**

100% local: no cloud calls, no telemetry egress — works in tunnels, workshops and test yards. Response, privacy and availability do not depend on connectivity.

EXPECTED OUTPUT 1 · HIGH-LEVEL SYSTEM ARCHITECTURE

Hardware and software components, integrated end-to-end



HARDWARE

Automotive ARM SBC or Raspberry Pi 5 + AI HAT · 2–8 GB RAM · NVMe/eMMC

OS & SAFETY

Linux or QNX · hypervisor-isolated AI partition · read-only bus taps

RUNTIME

llama.cpp (4-bit GGUF) · DuckDB + SQLite · containerised services

INTERFACES

SocketCAN · vsomeip · Vehicle HAL AIDL · ROS 2 / CARLA simulation

EXPECTED OUTPUT 2 · VEHICLE DATA COLLECTION STRATEGY

Capture → normalise → tag → retain, every source maturity-tagged

CAPTURE → NORMALISE → TAG → STORE

SocketCAN / vsomeip / AIDL / logd

Source	Collection interface	Maturity
CAN / CAN-FD	SocketCAN + DBC decoding at 500 kbps	protocol
SOME/IP services	vsomeip listener + ARXML schema	protocol
Android Vehicle HAL	AIDL property poll + event subscribe	file / log
QNX / infotainment logs	logd + journald forwarder	file
ADAS events & camera	ROS 2 topics; clips captured on event	simulated
Diagnostic logs (DTC/UDS)	ISO-14229 listener, ring buffer	protocol

Retention tiers — real-time + historical

- Hot: last 48 h in RAM/SQLite — live queries and fast path
- Warm: 4–6 weeks in partitioned Parquet on eMMC/NVMe — trends & anomalies
- Cold: zstd-compressed trip archives for forensics (months)
- ≈ 25–60 MB/day → ≤ 3.5 GB holds 60 days of history [Assumption → measured in Round 2]

Recreating driving scenarios & faults

- CARLA + SUMO drive cycles (city, highway, cold morning)
- Fault injection: DTCs, thermal derate, ADAS shadow events
- Recorded CAN / ROS 2 logs replayed with can-utils player
- 6 ground-truth scenarios with golden questions and answers

Data maturity tags

protocol = live bus decode · file = parsed log · simulated = CARLA/replay. Every dashboard and answer carries its tag — no simulated data is presented as measured.

Time alignment

One monotonic clock across sources (PTP/NTP disciplined); every event carries a ±30 s context window so “what happened before” queries are exact.

EXPECTED OUTPUT 3 · SLM / LLM SELECTION RATIONALE

Smallest model that clears the bar — quantised, permissive, CPU-first

Model	Params	Licence	RAM @ Q4_K_M	Verdict
Qwen2.5-1.5B-Instruct	1.5B	Apache-2.0	≈ 1.0 GB	SELECTED — 32k context, strong function calling
Qwen2.5-0.5B-Instruct	0.5B	Apache-2.0	≈ 0.4 GB	Fallback for the strict 2 GB configuration
Llama 3.2 1B Instruct	1B	Llama Community	≈ 0.8 GB	Comparison baseline in benchmarks
Phi-3.5-mini Instruct	3.8B	MIT	≈ 2.4 GB	Rejected — exceeds the 2 GB memory budget
Gemma 2 2B	2B	Gemma Terms	≈ 1.6 GB	Rejected — redistribution terms + budget

How the choice was made

- Score = answer quality on a vehicle QA set × licence fit ÷ RAM cost; Apache-2.0 allows redistribution in the product
- Runs CPU-only through llama.cpp (MIT) as 4-bit GGUF — no GPU/NPU dependency, deterministic packaging
- Model role is deliberately narrow: classify the question, shape the answer from the evidence package, never compute figures
- Grammar-constrained JSON output so the validator can parse, re-check and cite every claim
- All RAM numbers are calculated estimates from declared assumptions → measured on the reference board [Design / Target]

Why a small model is enough

- Numbers come from SQL/analytics, not from the LLM
- SLM only does intent, summarisation and phrasing
- 4-bit weights: 0.4–1.0 GB → fits 2 GB with OS
- Model file pre-loaded from disk — no network needed at boot

Guardrails on the model

- Answer must cite evidence rows; no evidence → explicit “not found”
- Refuses safety-adjacent advice; points to the manual/dealer
- Model bundle swappable over-the-air without retraining
- Upgradable tier (1.5B → 3B) on vehicle HPC

EXPECTED OUTPUT 4 · SEARCH & RETRIEVAL WORKFLOW

Computed first, written second, verified always



FAST PATH — exact lookup

- "when / show all / list" questions
- Indexed temporal + DTC search, model skipped
- Typically < 200 ms

ANALYTIC PATH — why / compare

- SQL, trend and anomaly engines compute deltas
- Ranks causal contributors with magnitudes
- Returns rows, not prose

NARRATIVE PATH — what happened

- Retrieval over event summaries + trip context
- Small vector index over local embeddings
- Feeds context window for the SLM

Worked example — end to end

Q: "Why has my driving range reduced over the last two weeks?"

Evidence: usable capacity −4.2% · mean consumption +9.8% · 3 cabin-preconditioning spikes on cold mornings · 1 tyre-pressure DTC — each with signal, value, timestamp

Answer: one-sentence cause + ranked contributors + trend chart + source chips; the validator re-derives every number or flags it

Latency: parse 50 ms · retrieve 300 ms · generate 900 ms · validate 150 ms ≈ 1.4 s [Target — measured in Round 2]

EXPECTED OUTPUT 5 · HISTORICAL DATA STORAGE APPROACH

Weeks of searchable history in a few gigabytes — all on the vehicle

Tier	Store	Window	Serves
Hot	SQLite + in-process cache (WAL)	last 48 h	live queries, fast path, dashboards
Warm	Parquet + DuckDB, partitioned by day	4–6 weeks	trends, anomalies, retrieval index
Cold	zstd-compressed trip archives	months	forensics, validation replay, export

Canonical schema (one model for every source)

- signals(event_time, source, signal_id, value, unit, trip_id) — columnar, sorted for range scans
- events(type, severity, source, window_start, window_end) — DTCs, ADAS disengagements, thermal warnings
- trips(trip_id, scene, distance, energy, tags) — city / highway / cold-start labelling
- notes(embedding, trip_id, text) — local embeddings for “what happened” semantic search
- Written once by the ingest pipeline; read-only for the AI layer

Storage math

- ≈ 60 signals @ 10 Hz + events ≈ 25–60 MB/day
- Delta + zstd compression ≈ 5–10×
- ≤ 3.5 GB holds 60 days on 16 GB eMMC
- [Assumption → verified on reference board]

Search layer (all local)

- DuckDB SQL over Parquet partitions
- Inverted event index (DTC / signal keywords)
- Small HNSW vector index for narrative search

Retention & privacy policy

- Rolling window, auto-pruned — no cloud offload
- Full wipe on owner/factory reset

EXPECTED OUTPUT 6 · EDGE DEPLOYMENT ARCHITECTURE

One container image, four rungs — CI VM all the way to vehicle HPC

1 DEV / CI
x86 VM · CARLA + vCAN replay · QEMU / Cuttlefish — iterate freely at zero hardware cost

2 REFERENCE BOARD
Raspberry Pi 5 (4–8 GB) + AI HAT + NVMe — where the 2 GB / 2 s targets are proven

3 VEHICLE
Automotive-grade ARM SoC (Linux or QNX) · optional NPU · hypervisor-isolated AI partition

4 VEHICLE HPC
Same OCI image · larger model tier (1.5B → 3B) · multi-ECU fan-out, fleet-wide rollout

Packaging & operations

- Single OCI container + systemd unit; signed OTA bundles for model, index and rules
- Config selects model tier per hardware class; rollback on failed health check; safety partition never updated by this path

Component	2 GB build	4–8 GB build
SLM (Q4_K_M)	0.4 GB (0.5B)	1.0 GB (1.5B)
KV cache + runtime	0.2 GB	0.4 GB
DuckDB + indexes	0.3 GB	0.4 GB
Ingest + normalisers	0.4 GB	0.5 GB
OS + services	0.6 GB	1.0 GB
Total (target)	≈ 1.9 GB	≈ 3.3 GB

Latency budget [Target]

parse 50 ms · retrieve 300 ms · generate 900 ms · validate 150 ms ≈ 1.4 s end-to-end, inside the 2 s constraint.

Fast path keeps the promise

Exact lookups skip the model entirely — typically < 200 ms, leaving the budget for the hard “why” questions.

EXPECTED OUTPUT 7 · USER INTERACTION · PRIVACY & SECURITY

Three modes, one answer contract, a hard safety boundary

DRIVER — head unit

Voice or typed question on the infotainment screen; short answer, trend sparkline, follow-up chips (“compare last month”, “what changed?”). No jargon, no raw signal names.

SERVICE ENGINEER — tablet

VIN-scoped session: DTC timeline, evidence rows, replay of the fault window; one tap exports the answer and its evidence into the repair order.

FLEET / VALIDATION — dashboard

Batch questions across vehicles, CSV/PDF export, scenario replay for regression runs — same engine, same citations.

Anatomy of an answer card

- Claim (one sentence, plain language)
- Evidence rows: signal · timestamp · value · delta
- Trend chart + source chips; confidence and honesty labels ([Target], [Assumption])

Safety boundary — non-negotiable

- Read-only taps: no CAN write, no actuator path
- AI partition isolated by hypervisor + MMU; safety functions bypass the AI entirely

Privacy & security

- Local-only processing: nothing leaves the vehicle
- Role-based access, encrypted at rest, query audit log

VALIDATION · ROADMAP · IMPACT

Targets we will measure — and the plan to hit them

Metric	Target	How it is measured
Query latency (common)	< 2 s	scripted 100-query set on the reference board
Fast-path lookup	< 200 ms	indexed temporal lookup benchmark
Memory footprint	≤ 2 GB	cgroup telemetry during the query loop
Retrieval accuracy	≥ 90%	6 ground-truth scenarios, golden answers
Answer factuality	100% validated	validator re-derives or flags every figure
History window	≥ 4 weeks	retention test over a synthetic 60-day log

Honesty labels used throughout: [Official] from the case brief · [Design] our architecture choice · [Assumption] declared input · [Target] to be measured · [Future] beyond this round.

No measured results are claimed yet. Every performance figure in this deck is a design target or a calculated estimate; the Round 2 prototype fills the table with real numbers.

Roadmap to the Round 2 demo

- M1 · Sim rig: CARLA + vCAN + ingest pipeline
- M2 · Tiered store, SQL/temporal/anomaly search
- M3 · SLM integration + evidence validator
- M4 · Head-unit HMI, 2 GB / 2 s tuning, benchmark run
- Demo: Pi 5 + AI HAT answering the 5 brief questions live

Innovation & practicality

- Works offline in yards, workshops and tunnels
- OTA-swappable model; plug-in normaliser per new source

Value: drivers get answers, engineers get evidence, fleets get trends, validation gets replay.